

Trajectoire Robot

CIEL 2

2026 - 2027

Polymorphisme



Notion de cours abordés :

- **Classes** : Création et utilisation de classes en C++.
- **Attributs privés et protégés** : Gestion de la visibilité et de l'accès aux attributs des classes.
- **Constructeur et Destructeur** : Initialisation et nettoyage des objets.
- **Méthodes de la classe** : Définition et utilisation des fonctions membres.
- **Mutateur** : Getter et Setter, méthodes utilisées pour accéder ou modifier les valeurs des attributs privés d'une classe.
- **Instance dynamique** : Création d'objets dynamiquement avec des pointeurs.
- **Flux d'entrée/sortie standard** : Utilisation de la sortie standard (cout).
- **Relation d'héritage** : Création de classes dérivées.
- **Fonction virtuelle** : Ce sont des fonctions membres d'une classe qui peuvent être redéfinies dans des classes dérivées. Elles permettent un comportement polymorphique dynamique.
- **Classe abstraite** : C'est une classe qui contient au moins une méthode virtuelle pure (méthode sans implémentation, déclarée avec = 0). Elle ne peut pas être instanciée directement.
- **Relation de composition** : Inclusion d'objets d'une classe dans une autre classe.
- **Gestionnaire de version (GitHub)** : Utilisation de Git pour versionner le code.
- **Documentation (Doxygen)** : Génération de la documentation du code à partir des commentaires.
- **Diagramme de classes** : Utilisation de Modelio pour réaliser le diagramme de classe de l'application.

A) CONTEXTE :

L'objectif de ce programme est de permettre la création, la mémorisation et la modification de la trajectoire d'un robot mobile au sein d'une entreprise. Cette trajectoire sera composée d'une liste d'éléments de différents types (tels que des segments, des arcs de cercle, etc.).

Votre travail doit être déposé dans votre repository GitHub dédié à l'apprentissage du C++. Ce repository servira à stocker et versionner votre code tout au long des différentes étapes du TD. Assurez-vous de nommer chaque commit de manière descriptive, en fonction de l'étape ou de la fonctionnalité implémentée.

Ressources utilisées :

Matériel

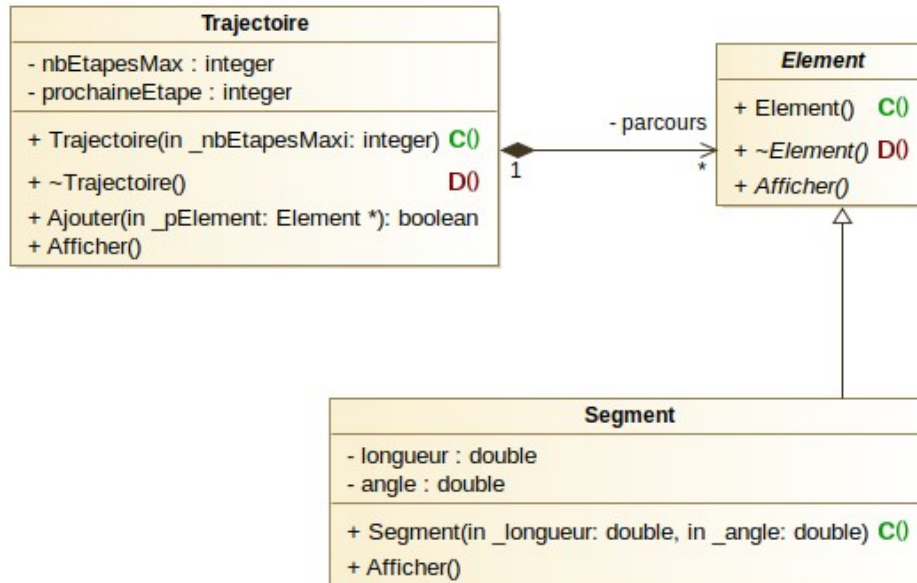
Un ordinateur sous Linux

Logiciel

Git
Doxygen
Qt Creator
Modelio

B) CRÉATION DES CLASSES DÉFINISSANT LA STRUCTURE DU PROJET

Dans un premier temps, le programme possède 3 classes comme le montre le diagramme de classe ci-après :



Description :

La classe nommée **Element** sert de classe de base pour les autres classes qui composeront la trajectoire. Elle contient un constructeur, un destructeur virtuel et une méthode virtuelle pure **Afficher()**.

Une classe **Segment** dérivée de **Element**. Un **Segment** est caractérisé par sa **longueur** exprimée en unité de longueur et son angle par rapport à l'horizontale exprimé en radian.

Une classe **Trajectoire** représente le conteneur des différents éléments composant la trajectoire. Elle dispose entre autres d'un tableau dynamique de pointeurs pour stocker les adresses de ces éléments.

- Comment se nomme la relation entre la classe **Trajectoire** et la classe **Element**, sous quelle forme cette relation doit être implémentée en C++ ?
- Comment peut-on qualifier la classe **Element** ? Justifier votre réponse.
- Dans un projet de type application C++ sans utiliser la librairie **Qt**, codez ces trois classes.
- Réalisez un premier programme principal permettant, à titre d'exemple, d'obtenir l'affichage suivant.

Réalisé dans la classe Trajectoire

Réalisé dans la classe Segment

```
Fichier  Editer  Affichage  Rechercher  Terminal  Aide
Trajectoire :
SEGMENT L = 9      A = 0
SEGMENT L = 5      A = 0.927295
Appuyez sur <ENTRÉE> pour fermer cette fenêtre...
```

- Déposez cette étape du projet dans votre **GitHub** avec comme message d'indexation « Étape 1 ».

C) COMPLÉMENT APPORTÉ AUX TROIS CLASSES PRÉCÉDENTES

6. Compléter la classe **Element** :

- Ajoutez un attribut protégé nommé `numero`, initialisé à 0 par le constructeur. Cet attribut sera incrémenté à chaque ajout d'un élément dans la trajectoire. Pour manipuler cet attribut, prévoyez des fonctions « Getter » et « Setter » associées.
- Ajoutez un second attribut protégé nommé `vitesse`, initialisé par le constructeur avec une valeur par défaut de 1.

7. Complétez le constructeur de la classe **Segment** afin qu'il tienne compte des ajouts précédents. Modifiez la fonction d'affichage de cette classe pour obtenir la figure suivante :

Affichage du
numéro dans la
trajectoire

→

Fichier Editer Affichage Rechercher Terminal Aide

Trajectoire :

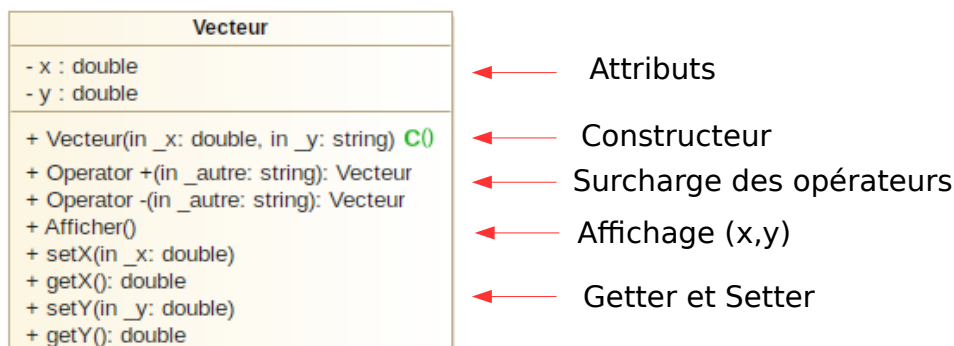
```
(1) SEGMENT L = 9      A = 0      V = 3
(2) SEGMENT L = 5      A = 0.927295 V = 1
```

Appuyez sur <ENTRÉE> pour fermer cette fenêtre...

←

Affichage de la
vitesse

8. Complétez le code de la méthode `Ajouter` de la classe **Trajectoire** afin de gérer l'incréméntation des éléments lors de leur ajout à la trajectoire.
9. Mettez à jour le code de la fonction `main` pour obtenir l'affichage souhaité en tenant compte des modifications précédentes.
10. Réalisez la classe **Vecteur** comme représentée sur la figure ci-après :



11. Ajoutez les fonctions virtuelles pures suivantes dans la classe **Element**, elles serviront de modèle pour les classes dérivées :

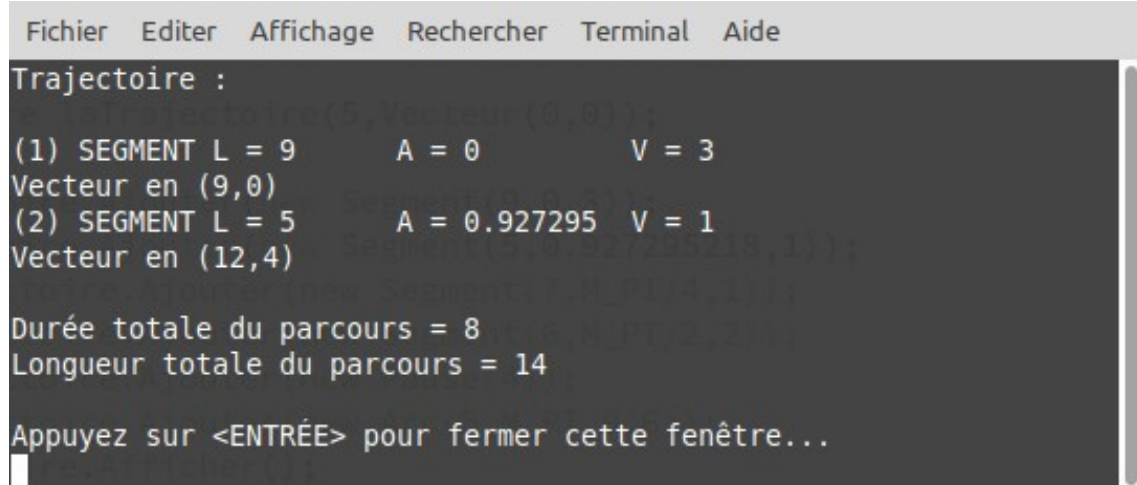
ObtenirLongueur()	Retourne la longueur de l'élément.
ObtenirDuree()	Retourne le temps nécessaire pour réaliser le mouvement de l'élément.
ObtenirVecteurArivee()	Retourne le vecteur résultant de l'élément.

12. Modifiez le constructeur de la classe **Segment** pour tenir compte de l'initialisation des nouveaux attributs de la classe **Element**, et ajoutez la surcharge des nouvelles méthodes pour compléter les fonctionnalités demandées..

Pour rappel :

le calcul du vecteur d'arrivée d'un segment est donné par les formules suivantes	
$x = \text{longueur} * \cos(\text{angle})$	$y = \text{longueur} * \sin(\text{angle})$
le temps pour réaliser le mouvement est donné par la formule	
$\text{durée} = \text{longueur} / \text{vitesse}$	

13. Mettez à jour l'affichage de la trajectoire pour intégrer les nouveaux éléments comme illustré dans la figure suivante. Le point de départ est supposé être à (0,0).

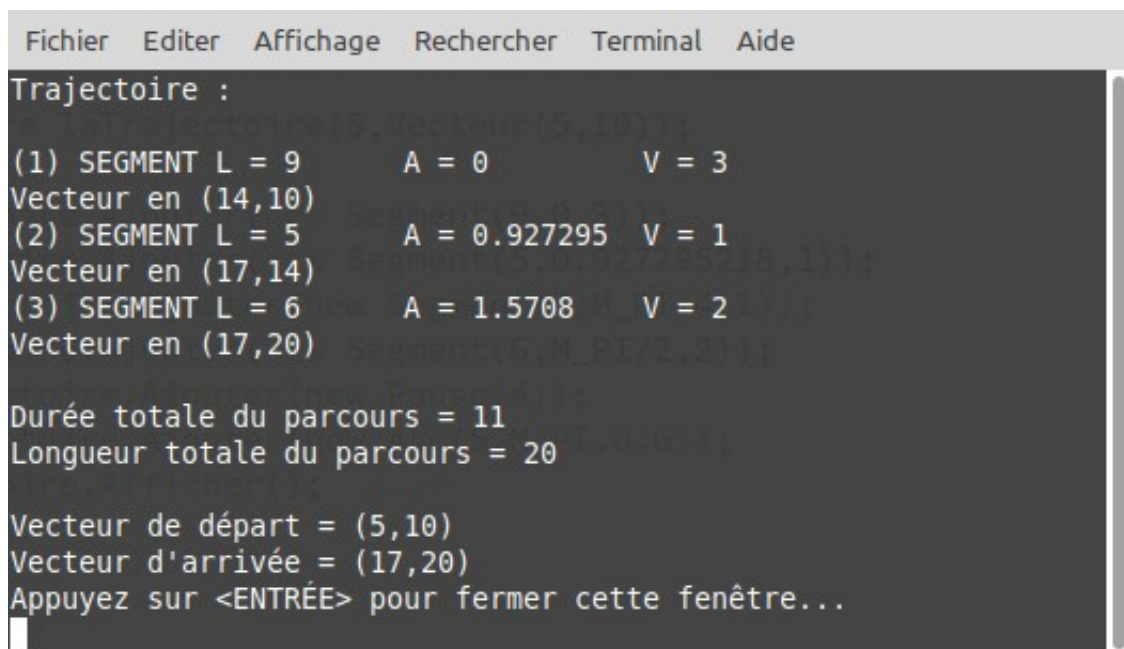


```
Fichier  Editer  Affichage  Rechercher  Terminal  Aide
Trajectoire :
(1) SEGMENT L = 9      A = 0      V = 3
Vecteur en (9,0)
(2) SEGMENT L = 5      A = 0.927295  V = 1
Vecteur en (12,4)

Durée totale du parcours = 8
Longueur totale du parcours = 14

Appuyez sur <ENTRÉE> pour fermer cette fenêtre...
```

14. Ajoutez un nouvel attribut **depart** de type **Vecteur** dans la classe **Trajectoire**. Cet attribut contiendra les coordonnées du point de départ et sera initialisé par le constructeur.
15. Mettez à jour la fonction d'affichage et le programme principal pour afficher une trajectoire avec les nouveaux éléments ajoutés.



```
Fichier  Editer  Affichage  Rechercher  Terminal  Aide
Trajectoire :
(1) SEGMENT L = 9      A = 0      V = 3
Vecteur en (14,10)
(2) SEGMENT L = 5      A = 0.927295  V = 1
Vecteur en (17,14)
(3) SEGMENT L = 6      A = 1.5708    V = 2
Vecteur en (17,20)

Durée totale du parcours = 11
Longueur totale du parcours = 20

Vecteur de départ = (5,10)
Vecteur d'arrivée = (17,20)
Appuyez sur <ENTRÉE> pour fermer cette fenêtre...
```

Déposez cette étape dans votre GitHub avec comme message d'indexation « feat : Classes Trajectoire, Element, Segment ».

D) CR ÉATION DE NOUVEAUX ÉLÉMENTS

16. Créez une nouvelle classe **Pause**, représentant un arrêt du mobile dans la trajectoire. Elle doit contenir une donnée membre **tempsAttente**, initialisée par le constructeur.

Affichage pour la classe Pause avec un temps de pause égale à 4.

```
Fichier  Editer  Affichage  Rechercher  Terminal  Aide
(1) SEGMENT L = 9      A = 0      V = 3
Vecteur en (14,10)
(2) SEGMENT L = 5      A = 0.927295  V = 1
Vecteur en (17,14)
(3) SEGMENT L = 6      A = 1.5708    V = 2
Vecteur en (17,20)
(4) Pause D = 4
Vecteur en (17,20)

Durée totale du parcours = 15
Longueur totale du parcours = 20

Vecteur de départ = (5,10)
Vecteur d'arrivée = (17,20)
Appuyez sur <ENTRÉE> pour fermer cette fenêtre...
```

17. Créez une classe Arc qui possède les données membres rayon, angleDebut, et angleFin, définis comme des réels doubles précisions. Les angles sont définis par rapport au cercle trigonométrique centré sur le centre de l'arc et sont exprimés en radians.

Pour la longueur de l'arc, on donne la formule suivante

$$\text{longueur} = |\text{angleDebut} - \text{angleFin}| * \text{rayon}$$

Utilisez la fonction `fabs ( )` de la librairie `<math.h>` pour calculer la valeur absolue.

On donne la méthode permettant d'obtenir le vecteur résultant à la fin de l'arc

```
Vecteur Arc::ObtenirVecteurArrivee()
{
    Vecteur ptCentre(rayon * cos(angleDebut), rayon * sin(angleDebut));
    Vecteur ptArrivee(rayon * cos(angleFin), rayon * sin(angleFin));

    return (ptArrivee - ptCentre) ;
}
```

18. Mettez à jour l'affichage et le programme principal pour obtenir le résultat final tel qu'attendu.

Quelques valeurs de PI	
M_PI	3,14159
M_Pi/2	1,5708
3 * M_PI/2	4,71239

```

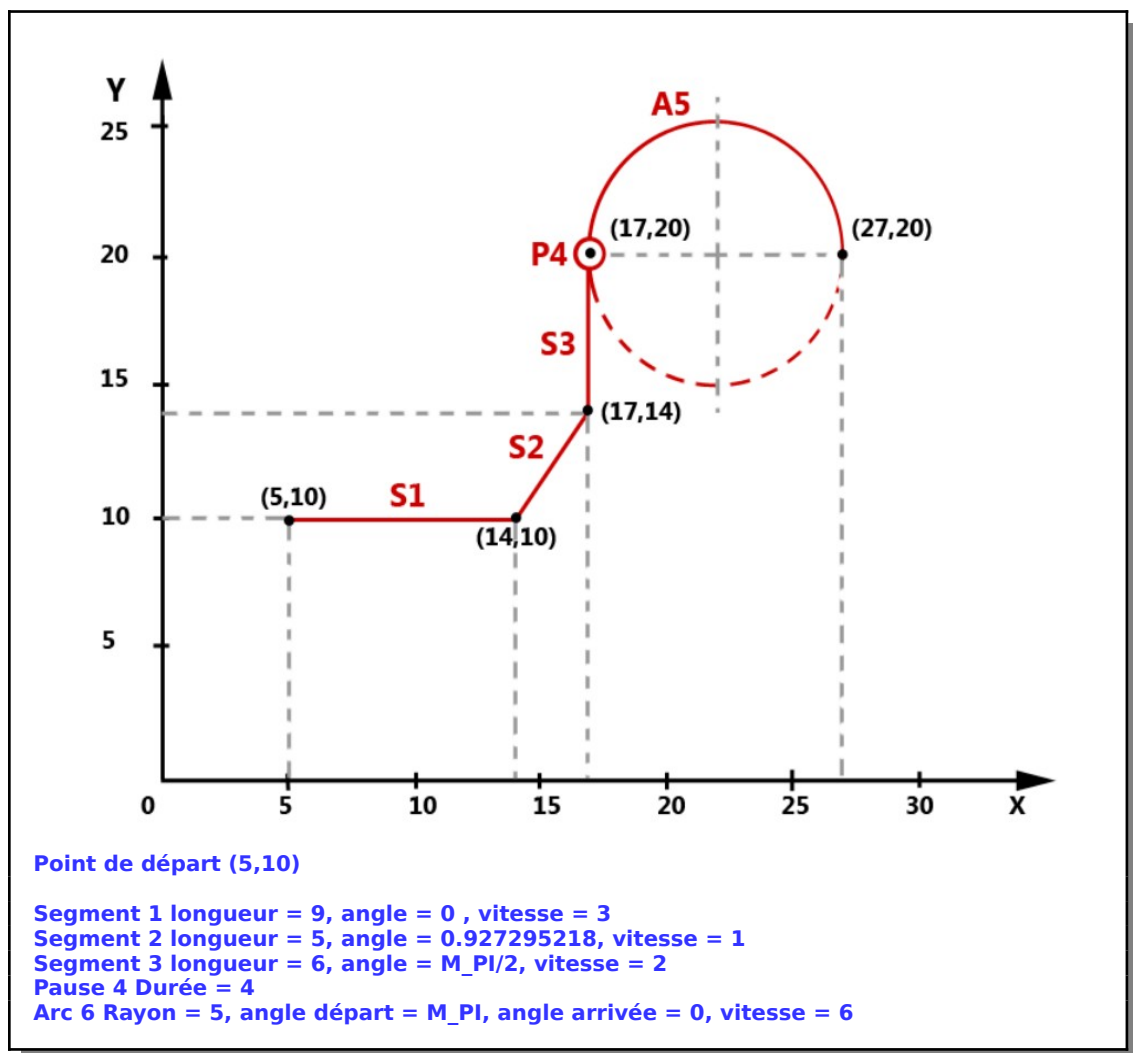
Fichier  Editer  Affichage  Rechercher  Terminal  Aide
Trajectoire :
(1) SEGMENT L = 9      A = 0      V = 3
Vecteur en (14,10)
(2) SEGMENT L = 5      A = 0.927295  V = 1
Vecteur en (17,14)
(3) SEGMENT L = 6      A = 1.5708    V = 2
Vecteur en (17,20)
(4) Pause D = 4
Vecteur en (17,20)
(5) ARC R = 5      Angle Debut = 3.14159  Angle Fin = 0      V = 6
Vecteur en (27,20)

Durée totale du parcours = 17.618
Longueur totale du parcours = 35.708

Vecteur de départ = (5,10)
Vecteur d'arrivée = (27,20)
Appuyez sur <ENTRÉE> pour fermer cette fenêtre...

```

Voici la figure correspondante à la trajectoire donnée en exemple :



19. Pour terminer, réalisez le diagramme de classe de votre application à l'aide de Modelio, enregistrez-le sous la forme d'une image png et insérez-la dans le fichier Readme de votre GitHub pour documenter la structure de votre application. Mettez à jour votre dépôt GitHub avec cette version finale en utilisant le message de commit « feat : version finale ».